SNUG PROTOCOL

The Snug Protocol: An Open Protocol for Agent-Backed Personal Software

A specification of the wire protocol, security model, and portable data format for sandboxed micro-applications that use a host's language model as a runtime capability — and that their users own outright, as a single file.

AUTHOR

Jeetu Maker

SPECIFICATION

v0.1 · v0.2 (draft)

DATE

August 2026

Abstract. Large language models have collapsed the cost of authoring software to the point where an application built for a single person is economically trivial. What has not existed is the structure that makes such software safe to run, durable enough to keep, and portable enough to own: an isolation model, a data format, and a wire contract that any vendor can implement. This paper specifies the Snug Protocol, which supplies that structure in two coupled layers. The first is a versioned postMessage frame protocol between a sandboxed application and its host, together with a tagged chat envelope by which the host relays an application's request to its own agent endpoint; this makes the host's model a runtime capability the application invokes rather than a code generator that produced it. The second is a portable user database — one SQLite file holding an individual's applications, their versions, their data as native tables, their conversations, and their settings — which travels intact between hosts, model providers, and devices. We present the protocol's normative rules, an explicit threat model in which application code is treated as hostile at all times, and the two hard constraints from which the security properties follow: credentials never cross into the application sandbox, and the sandbox profile is never weakened. We situate the design against the Model Context Protocol, vendor application platforms, and the local-first literature, and state its present limitations candidly.

Contents

1 Introduction 3
The market of one, and what it still lacks

2 Design goals and non-goals 4
What the protocol commits to, and what it deliberately refuses

3 Architecture: the three actors 5
Model provider, host provider, and the user who owns the file

4 The wire protocol 7
Frames, message types, normative rules, and the chat envelope

5 Security model 12
Threat model, the two hard constraints, and residual risk

6 The portable user database 16
One file: hub tables, native per-app data, sync origins

7 Conformance 20
What an implementation must do to claim the name

8 Related work 21
MCP, vendor application platforms, local-first software

9 Limitations and future work 23
What is unsolved, stated plainly

10 Conclusion 24

— Normative references 25

A Appendix: message schema summary 26

1 Introduction

For most of the history of commercial software, an application existed because someone judged that enough people wanted it. That judgement was an economic filter, and it was ruthless: if your need was too particular — too specific to your household, your condition, your one strange workflow — no product was built, because the addressable market was one person.

Language models dissolved that filter. The marginal cost of producing a small, working application has fallen far enough that building one for a single user is now unremarkable. Anyone who has asked an assistant for a bespoke tracker, a calculator shaped to their own rules, or a game invented by their child has produced software that no company would ever have shipped.

What did not arrive with that capability is the surrounding structure. Generated applications land as artifacts inside a vendor's product: they run under that vendor's rules, their data lives in that vendor's storage, and they are portable only in the sense that text is portable. The moment such an application needs to persist state, hold a credential, or outlive the conversation that produced it, three questions become urgent, and none of them is a modelling question:

- **Isolation.** The code was written by a language model, possibly under the influence of text the user did not author. On what terms may it run?
- **Custody.** Where does its data live, who can read it, and what happens to it when the user leaves?
- **Continuity.** If the application is genuinely the user's, it must survive a change of host, a change of model provider, and the disappearance of either.

These are protocol questions. They are answered once, in public, by a contract that multiple implementations can satisfy — or they are answered privately and differently by every vendor, which is the same as not being answered at all.

The Snug Protocol is a proposed answer. It rests on one architectural observation: an application does not need to *contain* intelligence if it can *call* it. A chess application built on Snug ships no chess engine. It sends the board position across a boundary, and the host's agent replies with a move. The application supplies the interface, the rules, and the state; the model supplies judgement, on demand, as a runtime capability. We refer to this throughout as the body/mind split, and it is a relationship at runtime, not a fact about how the code was produced.

That single decision reorganises everything else. If the model lives outside the application, the application needs no network access, and can therefore be sandboxed far more tightly than a conventional web page. If it needs no network access, it cannot exfiltrate a credential, so credentials can be kept strictly on the host side of the boundary. And if the application is a small self-contained program with its data held separately, then the application and its data can be written into a single portable file that the user carries between hosts.

The protocol has two layers, specified in this paper. Section 4 covers the wire protocol: nine versioned post-Message frames and a tagged chat envelope, normative at v0.1. Section 6 covers the portable user database, a storage and host-behaviour layer that leaves the wire protocol untouched and is published as a v0.2 draft.

Section 5, which sits between them, is in our view the load-bearing part: the threat model and the two hard constraints that the other two layers exist to preserve.

> **CONVENTIONS**
>
> The key words **MUST**, **MUST NOT**, **SHOULD**, and **MAY** are used in the sense of RFC 2119. Material drawn from spec v0.1 is normative. Material drawn from the v0.2 draft is marked DRAFT and is published for review, not for conformance. Where this paper and the specification differ, the specification governs.

2 Design goals and non-goals

A protocol is defined as much by what it declines to do as by what it specifies. We state both here so that later sections can be read as consequences rather than choices.

2.1 Goals

G1 The agent is a capability, not a dependency

An application MAY call the host's model at runtime; it MUST NOT be required to. An arcade game whose rules are entirely deterministic should compute locally, pay no latency, and run with no model configured at all. The host advertises what it can do and degrades gracefully when an application asks for nothing.

G2 Hostile-by-default application code

Application code is authored by a language model, which may have been influenced by content the user did not write. The protocol therefore assumes the application is hostile and derives its guarantees from isolation, not from the good behaviour of generated code.

G3 The user owns the artifact

Everything an individual accumulates — applications, versions, data, conversations, settings — is one file in a standard, inspectable format they can export at any time and open with ordinary tools.

G4 No required backend

A conforming host is deliverable as static files. A hosted service may add convenience — synchronisation, a subscription path to a model — but application execution MUST NOT depend on it.

G5 Additive evolution

Unknown message types and unknown fields are ignored rather than rejected, so that a newer participant can talk to an older one without negotiation beyond a version check.

G6 Implementable by more than its author

Every message is published as JSON Schema, exported byte-identically from the reference implementation. The schemas, not the prose, are the contract.

2.2 Non-goals

- **A marketplace.** The protocol says nothing about discovery, distribution, ratings, or payment. It specifies how an application talks to a host, and what a user's file contains.

- **A general plugin system for agents.** Extending what a model can *do* — tools, resources, external systems — is well served by the Model Context Protocol, and Snug does not duplicate it. Snug addresses the inverse direction: giving a user-authored application access to the model. Section 8 develops the distinction.
- **A rendering or component standard.** An application is an ordinary self-contained HTML document. The protocol constrains what crosses the boundary, not what the application draws.
- **Cryptographic custody.** The protocol does not claim that a host is incapable of reading a user's secrets — only that a conforming host never receives them. The distinction is drawn precisely in §5.4, and we regard overstating it as a security defect in its own right.

3 Architecture: the three actors

Snug models three parties, deliberately kept separable, as shown in Figure 1. The separation is the portability story: any one of them can be replaced without disturbing the other two.

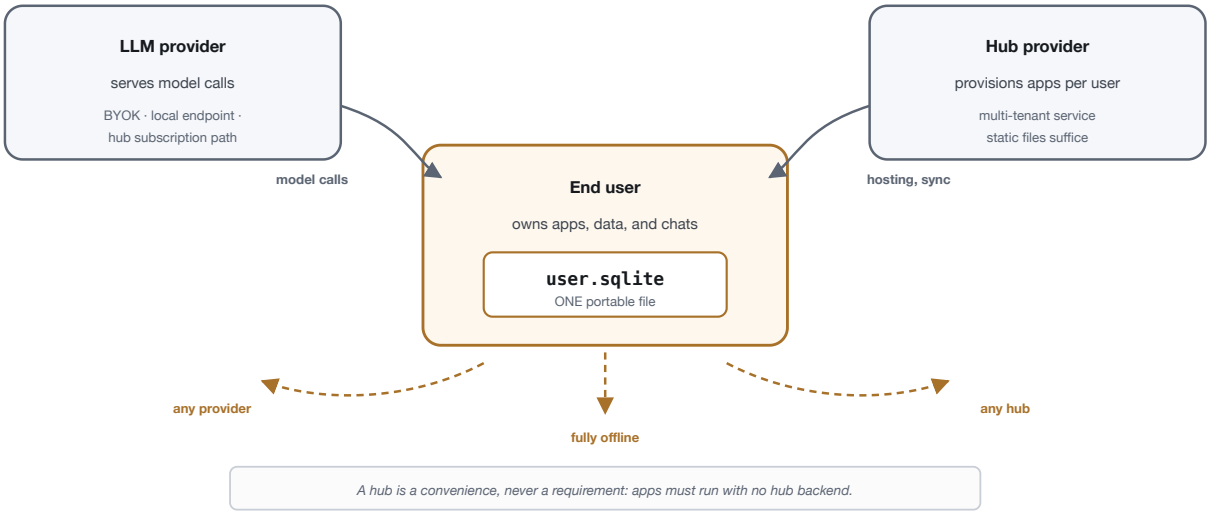

Figure 1. The three actors. The user's file is the centre of gravity: it holds everything the user has accumulated and can move to any model provider, any host, or none. A host provider is a convenience — the protocol requires that applications run without one.

The **model provider** serves inference. It is reached in one of three ways — directly from the browser with a key the user supplies, directly from the browser against a local endpoint on the user's own machine, or through a host's subscription path — and §6.5 treats all three as a single settings choice rather than a migration.

The **host provider** operates the surface where applications are built and run. It provisions applications per user and may offer synchronisation, caching, and a subscription path to a model. What it may not do is make itself necessary: a conforming host delivers application execution from static assets, and its server components, where they exist, are strictly optional.

The **end user** owns the artifact. Not an account, not rows in a vendor's database — one file, in a format with a thirty-year record of stability and universal tool support.

Figure 2 shows how these actors are realised inside a running host. Two boundaries in that diagram carry the security properties, and both are examined in §5: the seam between the host page and the sandboxed frame, and the point at which an application-originated request is validated before it can have any effect.

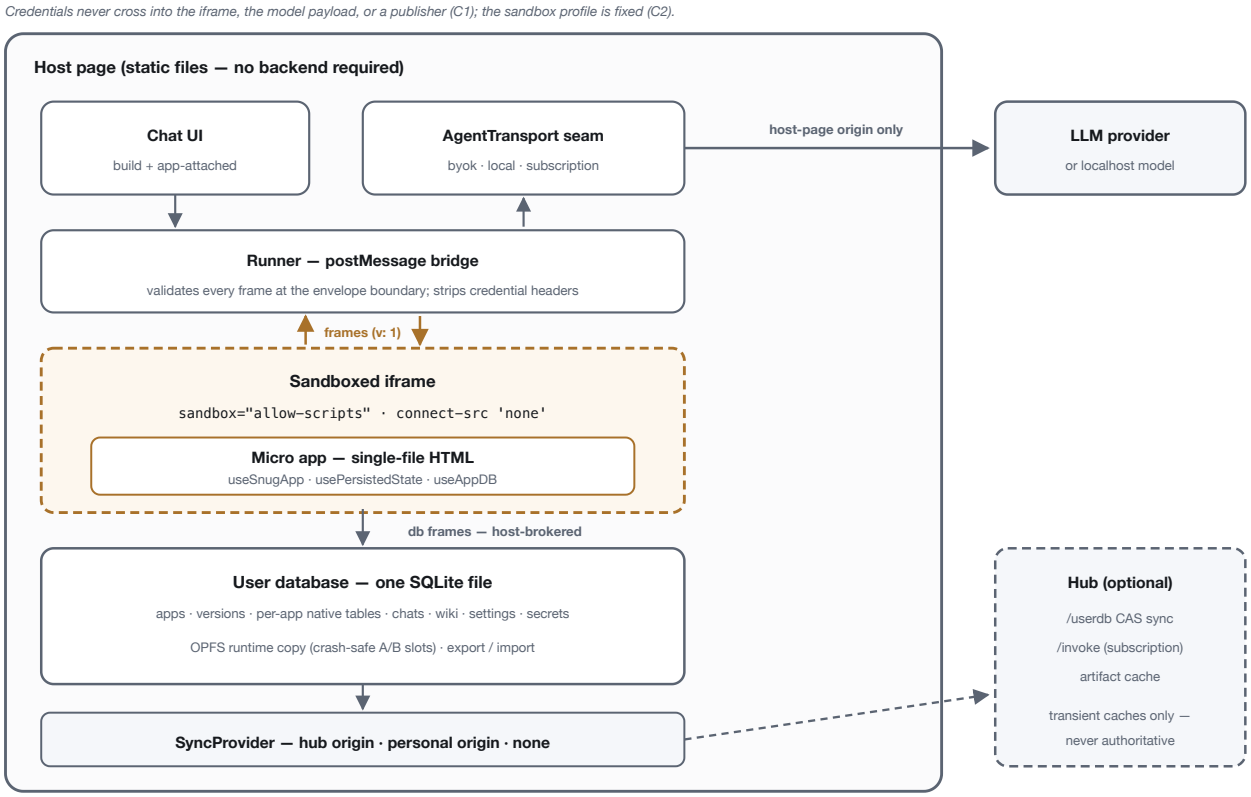

Figure 2. System architecture. The application is confined to a sandboxed frame with no network egress; every capability it has — model access, storage, connectivity — is mediated by the host through versioned frames. Model calls originate from the host page only. The user database is authoritative in every mode; host-side stores are transient caches.

3.1 The runtime relationship

The consequence of G1 and G2 together is that the application is a body and the agent is its mind, but the body is fully capable on its own. Two applications illustrate the poles. A chess opponent calls the model on every move: the judgement it needs — what is a good move — is not expressible as a rule the application could apply itself. An arcade game calls the model never: collision detection, scoring, and win conditions are deterministic, and routing them through a model would add latency, cost, and a dependency on a configured key to a program that should run offline.

Both are conforming Snug applications. Deciding which turns need the model is an authoring decision, and the protocol records nothing about it — the property is emergent from the application's code, and the host already advertises its capabilities and degrades when they go unused.

4 The wire protocol NORMATIVE · V0.1

The wire protocol comprises two coupled contracts. *Frames* are postMessage messages exchanged between the application frame and the host runner. The *chat envelope* is the tagged message a host sends to its own

agent endpoint on behalf of an application, together with the reply contract the agent must satisfy.

4.1 Frames

Nine frame types are defined. Each carries `v: 1`, and each is published as a JSON Schema exported byte-identically from the reference implementation.

TYPE	DIRECTION	PURPOSE
`snug:app-announce`	app → host	Self-describing metadata on mount: `appId`, `displayName`, `description`, `iconEmoji`, `iconColor`. The host acknowledges with `snug:host-ready`.
`snug:host-ready`	host → app	Sent on frame load and again as an announce acknowledgement (idempotent): `instanceId`, `protocolVersions`, `capabilities` (streaming, db, auth), `theme`, optional `locale`.
`snug:app-message`	app → host	A request for the agent: `requestId`, `instanceId`, `appId`, `action`, and optional structured `payload`, `state`, and `responseSchema`.
`snug:app-cancel`	app → host	Abort an in-flight `requestId`.
`snug:app-response`	host → app	Streaming, final, or error, per R3. Three shapes: cumulative `text`; a final `data` payload; or an `error`.
`snug:db-request`	app → host	Host-brokered per-application storage. Operations: `exec`, `export`, `import`, `kvGet`, `kvSet`.
`snug:db-response`	host → app	Result rows and columns, an exported byte payload, or an error.
`snug:host-event`	host → app	Open additive channel: `theme-change`, `visibility`, and further namespaced events. Unknown events are ignored.
`snug:app-event`	app → host	The reverse channel, e.g. `resize` carrying a height.

Table 1. The nine frame types of spec v0.1. Schemas are input shapes: validators MUST accept unknown fields (R2).

Storage is brokered rather than direct for a reason that recurs throughout the design. An application frame runs with a null origin (§5.3), and browser storage APIs are unavailable to it. One of the production systems that preceded this protocol taught applications to use `localStorage`, which worked only because its sandbox was weakened enough to permit it — precisely the weakening this specification forbids. Routing storage through frames preserves the sandbox and gives the host a mediation point.

4.2 The application lifecycle

Figure 3 traces a complete request, including the two paths by which a request may end without a normal answer.

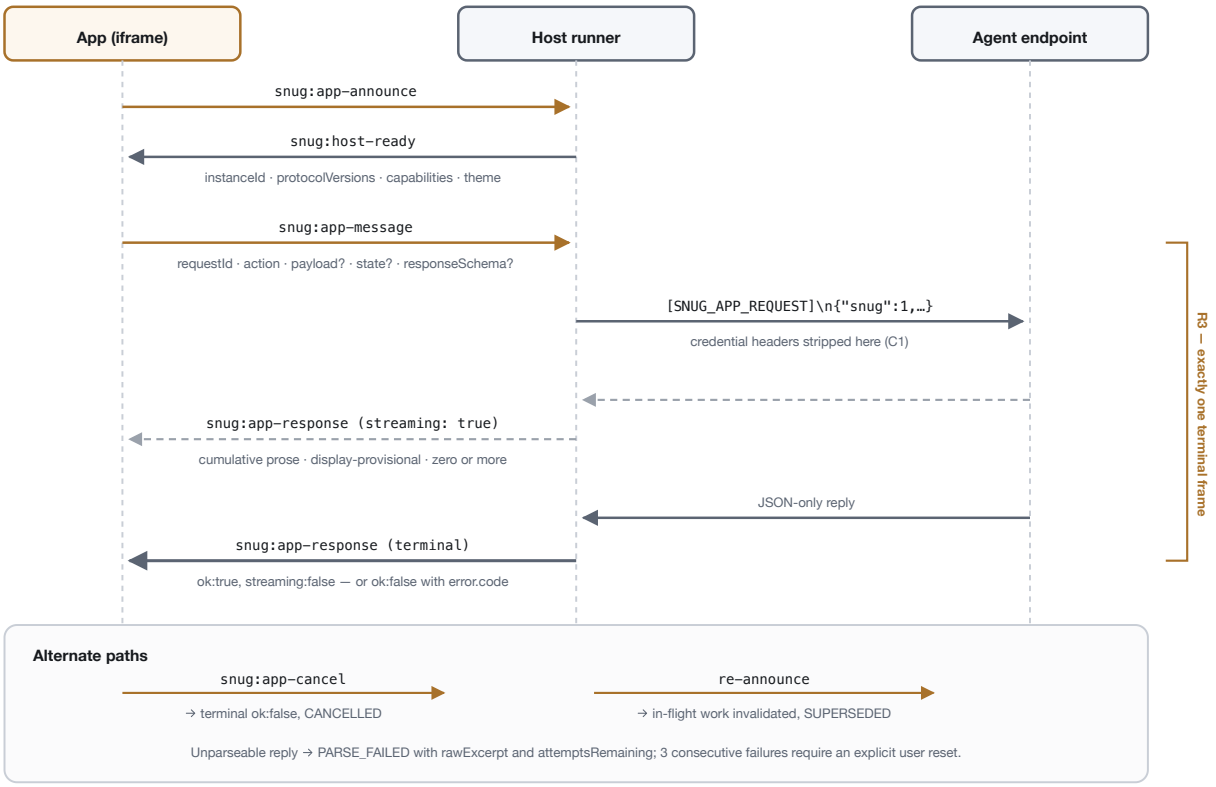

Figure 3. The frame lifecycle. The application announces itself and receives an instance identity; requests then flow outward through the host, which strips credential headers before relaying an envelope to the agent endpoint. Streaming frames are provisional; exactly one terminal frame is authoritative (R3).

4.3 Normative rules

Six rules constitute the normative core of v0.1. We give each rule and, briefly, the failure it exists to prevent — since a rule whose motivation is undocumented tends to be implemented approximately.

R1 Versioning

Every frame carries `v: 1`; the chat envelope carries `snug: 1`. Unsupported versions are rejected with `UNSUPPORTED_VERSION`. Parse failures surface a `requestId` recovered from the raw frame when it carried a plausible string identifier; hosts answer `UNSUPPORTED_VERSION` or `MALFORMED` on the wire only in that case, and never otherwise. Answering an unidentifiable frame would mean emitting a reply that no request can be correlated with — an invitation to a spurious response being attributed to an unrelated request. `snug:host-ready.protocolVersions` advertises what the host supports.

R2 Additivity

A frame with a valid `v` but an unrecognised `snug:*` type MUST be silently ignored, and unknown fields on known frames MUST be ignored. The `snug:` type prefix and the event namespaces are reserved. This is what allows a host and an SDK to be upgraded independently.

R3 Terminal frame

Every accepted `requestId` receives exactly one terminal `snug:app-response`, either successful and non-streaming or an error. `streaming: true` frames are cumulative prose and are display-provisional; the terminal frame is authoritative. Hosts MAY suppress streaming for schema-constrained requests.

Without this rule an application cannot know when to release a pending interaction, and a dropped final frame is indistinguishable from a slow one.

R4 Identity

Hosts route by `event.source`. A sandboxed frame has a null origin, so `targetOrigin` is necessarily `'*'` and origin comparison is unavailable as an identity mechanism — the message source object is the identity. The host mints `instanceId` and delivers it in `snug:host-ready`; applications echo it in every request. A fresh `snug:app-announce` from the same frame invalidates in-flight work with `SUPERSEDED`. `appId` is display metadata and **not a security principal**: it is self-asserted by the application and MUST NOT be used to make an access decision. `requestId` MUST be unique per instance.

R5 Error codes

`error.code` is an open string. The known codes are `PARSE_FAILED`, `THREAD_CONFLICT`, `NETWORK_ERROR`, `RESET_FAILED`, `CANCELLED`, `SUPERSEDED`, `UNSUPPORTED_VERSION`, `CONSENT_REQUIRED` (reserved), `AUTH_REQUIRED` (reserved for a future authenticated-connections revision), and `HOST_ERROR`. `MALFORMED` is defined by R1 as a wire answer to an unparseable frame and is not part of this list. Receivers treat unknown codes according to the `retryable` flag and render them as `HOST_ERROR`.

R6 Limits

Frames are capped at 256 KiB, except `db-request` and `db-response`, which occupy their own size class at 8 MiB so that a base64-encoded 5 MiB artifact round-trips through the storage bridge; artifacts are capped at 5 MiB, `rawExcerpt` at 200 chars, and announce strings at displayName 80 and description 400. The parse-failure budget is 3 consecutive failures, after which the host requires an explicit user reset. Thread-conflict backoff is 100 / 250 / 500 ms.

> **WHY A PARSE-FAILURE BUDGET**
>
> A model that returns malformed output once will often do so repeatedly for the same input. Unbounded retries turn a formatting failure into a billing incident and a denial of service against the user's own quota. Three strikes, then a human decision.

4.4 The chat envelope

When an application requests the agent, the host does not expose the model endpoint to it. The host constructs an envelope and sends it to its own agent endpoint, using the same path its ordinary chat traffic takes. The wire form is a tag line followed by a JSON object:

```
// The host sends this to its own agent endpoint — never the application.
[SNUG_APP_REQUEST]
{
  "snug": 1,
  "appId": "portfolio-tracker",
  "instanceId": "c0a8-…",     // host-minted; the routing identity
  "requestId": "9f1c-…",      // unique per instance
  "action": "suggest_rebalance",
  "payload":   { /* structured, application-defined */ },
  "state":     { /* the application's own view of context */ },
  "responseSchema": { /* optional: constrain the reply shape */ }
}
```

Detection requires both the tag prefix and the `snug: 1` marker. Requiring two independent signals prevents ordinary chat content that happens to quote the tag from being processed as an application request.

Servers SHOULD skip thread history for application requests: the envelope is self-contained by way of `state`, and replaying an unrelated conversation into an application's turn both costs tokens and leaks the user's chat into a context the application can observe through the reply. Servers MUST apply the credential-header strip described in §5.2.

4.5 The agent reply contract

The agent replies with only a JSON object; a human-readable `message` field is recommended. Because models emit fenced code blocks and surrounding commentary with some regularity, hosts parse with graduated tolerance — a raw parse first, then a fenced block, then balanced-object extraction — and reject null, array, and scalar replies. A failure at every stage becomes a `PARSE_FAILED` frame carrying `rawExcerpt` (capped at 200 chars) and `attemptsRemaining`, so the application can surface something honest rather than hanging.

Tolerant parsing here is not indiscipline. The alternative — failing the turn on a stray code fence — converts a cosmetic model artifact into a broken application, and every implementation would end up writing the same three fallbacks privately anyway. Specifying them makes behaviour uniform across hosts.

5 Security model NORMATIVE · V0.1

This section states what the protocol defends against, the two constraints from which its guarantees derive, and — with equal prominence — what it does not defend against.

5.1 Threat model

The adversary is **the application itself**, and Figure 4 shows exactly where that adversary is confined. This is not a hypothetical posture. Application code is authored by a language model from a prompt that may incorporate untrusted text: a web page the model consulted, a document the user pasted, a message from a third party. Prompt injection is a live technique with no complete defence at the model layer, so the protocol assumes injection sometimes succeeds and constrains what a successful injection can accomplish.

THREAT	MECHANISM	RESIDUAL
Application exfiltrates a credential	Credentials never enter the frame; `connect-src 'none'` removes every egress path the application could use	None by construction, provided C1 and C2 hold
Application reads another application's data	Runtime isolation is physical: an application executes against a materialised database containing only its own objects (§6.3)	Host implementation defect
Application reads host-namespace tables	Reserved prefixes are refused; unvalidated names are never interpolated, and a violation fails the write closed	Host implementation defect
Application escalates by asserting another identity	Routing is by message source, not by any self-asserted field; `appId` is explicitly not a principal (R4)	None at the protocol layer
Injected instructions cause a privileged call	The application has no privileged call available: capabilities are host-mediated and credential injection is host-bound and always strict	Model-layer manipulation of answer *content* (§5.4)
Application exhausts the user's model quota	Parse-failure budget of 3; cancellation; host-side rate control	An application may still make many valid requests
Malformed frames drive the host into an inconsistent state	Schema validation at the boundary; exactly one terminal frame per accepted request (R3); bounded backoff	Host implementation defect

Table 2. Threats and the mechanism that answers each. Every mechanism is structural; none depends on the application behaving correctly.

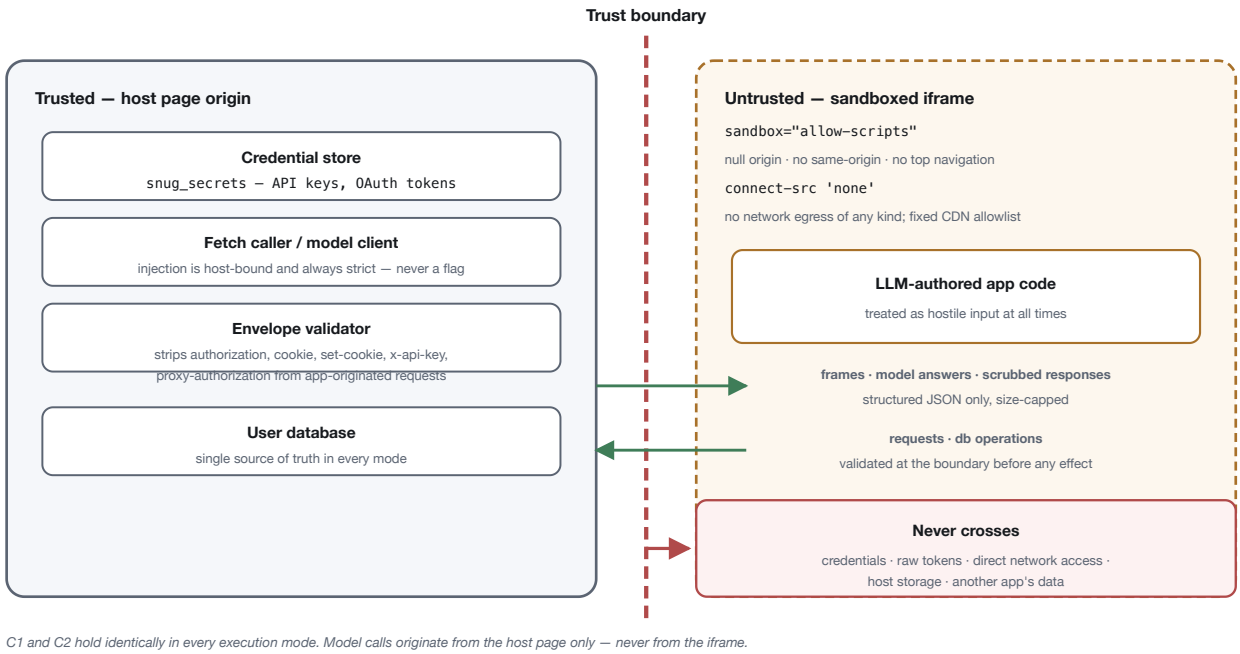

Figure 4. The trust boundary. Structured, size-capped JSON crosses in both directions and is validated before it can have any effect. Credentials, raw tokens, direct network access, host storage, and other applications' data never cross at all.

5.2 C1 — the token boundary

The first hard constraint: **credentials never enter the application frame, never reach the model, and never reach a publisher.**

Concretely, hosts MUST strip `authorization`, `cookie`, `set-cookie`, `x-api-key`, and `proxy-authorization` from any application-originated request at the envelope boundary. That header strip is the whole of C1's normative wire surface at v0.1.

Beyond it, the reference implementation adopts a rule we recommend to any host and expect a later revision to make normative: where a host injects a credential on an application's behalf, injection is host-bound and **always strict — never a configurable flag**. It is stated here as implementation doctrine, not as a v0.1 requirement, because the authenticated-connections surface it governs is itself unpublished (§9).

The emphasis is earned. An audit of the two production systems this protocol generalises from found a strictness control that defaulted to off, which is to say a documented feature that silently converted a prompt injection into credential exfiltration. A security property that can be disabled by configuration is a security property that will be disabled in some deployment, and the deployment where it is disabled is the one that gets attacked. The protocol therefore does not offer the choice.

5.3 C2 — sandbox integrity

The second hard constraint: **application frames run with `sandbox="allow-scripts"` only, with `connect-src` blocked.** These are never weakened, including for demonstrations. Hosts that permit any script source at all SHOULD confine it to a fixed allowlist, as the reference implementation does; the allowlist itself is a host policy rather than a v0.1 requirement.

Three properties follow. First, the frame is assigned a **null origin**: `allow-same-origin` is absent, so the application cannot reach host storage, host cookies, or the host DOM — and, as noted in R4, this is also why identity must be established by message source. Second, `connect-src 'none'` removes fetch, XHR, WebSocket, and EventSource, so there is no channel by which data could leave even if the application obtained something worth sending. Third, because ordinary storage is unavailable to a null origin, persistence must be brokered by the host, which is what makes per-application isolation enforceable rather than advisory.

> **A NOTE ON DEMONSTRATION BUILDS**
>
> The most common way a sandbox weakens is a temporary exception that is never removed. The reference implementation asserts the sandbox profile in its test suite: a frame carrying `allow-same-origin` fails the build. We recommend the practice to any implementer — a constraint that is only documented is a constraint that erodes.

5.4 Residual risks

We state these plainly, because a threat model that lists only solved problems is marketing.

Secrets are persistent at rest. DRAFT In the draft storage layer of §6, credentials live in the `snug_secrets` table inside the user's own file. They are stripped from host-bound synchronisation and from default exports, and the file is vacuumed so that freed pages retain nothing recoverable. They are nonetheless present in the local file. This is a deliberate **trade-off** in favour of portability: cross-device continuity means the credentials travel with the file to storage the user chooses. No key-management or host-blind claim is made, and none should be made until a key-provider layer ships.

The honest formulation, and the only one we endorse: *your keys never reach our servers; your file, including keys, goes only to storage you choose.* That is a statement about host custody. It is not the stronger claim that keys never leave the local file, which would be false the moment a user connects a personal synchronisation origin — a legitimate and intended use.

The model can be manipulated in what it says. C1 and C2 bound what an injected instruction can *do*: it cannot reach a credential, open a socket, or touch another application's data. It can still influence the *content* of an answer the application then renders. Applications that act on model output — rather than display it — should treat that output as untrusted input, and hosts should prefer `responseSchema` so that replies are shape-constrained.

Host implementation defects remain possible. Several mechanisms above depend on the host implementing them correctly: name validation, materialisation, header stripping. The protocol can specify these and the schemas can constrain the wire, but neither can compel a host to be correct. This is the ordinary condition of protocols, and it is why §7 states conformance in terms that are testable.

Availability is not addressed. A misbehaving application can make many individually valid requests. Rate control is left to hosts, which are better placed to know their own cost model.

6 The portable user database DRAFT · V0.2

> **DRAFT STATUS**
>
> This section describes spec **v0.2**, published for review and **not yet normative**. It adds a storage and host-behaviour layer; the wire protocol of §4 is unchanged by it. A later revision may alter details described here. Storage schema version 2 is described; the source of truth for constants and DDL is the reference implementation, locked by a snapshot test.

If applications are the user's, their data must be too — and in a form that outlives any particular host. Snug's answer is a single SQLite file per user, `user.sqlite`, which is the canonical artifact. `PRAGMA user_version` carries the schema version; migrations are forward-only; total size is bounded by `MAX_USERDB_BYTES`, 64 MiB in the current draft. Figure 5 shows what that file contains.

SQLite is chosen for unglamorous reasons: it is a stable, universally supported, single-file format with three decades of tooling. A user who exports their file can open it in any SQLite browser and read their own data without our software. We consider that property, rather than any feature, the substance of ownership.

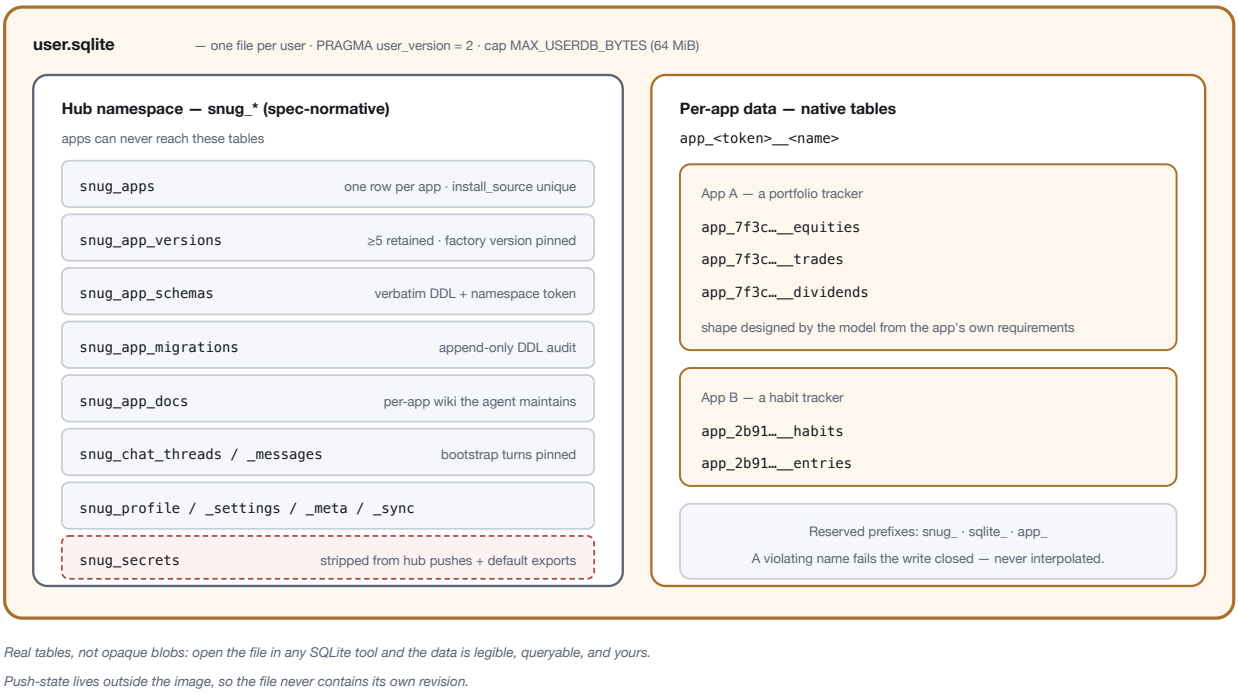

Figure 5. The user database. Host-namespace tables carry applications, their retained versions, their schemas and migration history, their documentation, and the user's conversations and settings. Application data sits alongside as real, legible tables under an injective namespace token.

6.1 Host-namespace tables

Tables prefixed `snug_` are the host namespace and are unreachable from applications. They hold the file's identity and creation metadata (`snug_meta`), the display profile (`snug_profile`), execution settings (`snug_settings`), credentials (`snug_secrets`, §5.4), one row per application (`snug_apps`), complete HTML per version (`snug_app_versions`), the schema registry (`snug_app_schemas`), an append-only DDL

audit (`snug_app_migrations`), a per-application documentation wiki (`snug_app_docs`), conversation history (`snug_chat_threads` and `snug_chat_messages`), and synchronisation configuration (`snug_sync`).

Two version-retention details are worth drawing out, because they encode a view of what an application is. Hosts retain at least `VERSIONS_RETAINED` — five — unpinned versions per application, pruning oldest first; but the *factory* version, the first build or the state at installation, is pinned and never pruned. Reverting is not deletion: it copies the target forward as a new version. The user can therefore always return to a working application, and the history of how it got there is not destroyed by the act of going back.

The documentation wiki deserves a note. Each application may accumulate markdown documents — advisory slugs include `vision`, `requirements`, `plan`, `lessons`, `memory`, and `next-tasks` — maintained by the agent as the application evolves. An application is thus not a frozen artifact but a living one, carrying its own rationale forward so that a later modification is informed by earlier intent.

6.2 Per-application data as native tables

Each application's data lives as **real tables** in the same file, named `app_<token>__<name>`. The token is produced by a normative, total, and injective function of the host-assigned namespace: a UUID-shaped namespace becomes 32 lowercase hex characters with dashes stripped, and anything else becomes `'x'` followed by the hex encoding of its UTF-8 bytes. Injectivity is what makes namespaces non-colliding; the `'x'` prefix sits outside the hex alphabet, so the two ranges cannot overlap.

The naming rules are normative. Reserved prefixes — `snug_`, `sqlite_`, and `app_`, case-insensitively — are refused, blocking an application from forging another application's at-rest name. Object names must match `^[A-Za-z][A-Za-z0-9_]{0,40}$`. The single exemption is the driver-internal `snug_kv` table, held at rest as `app_<token>__snug_kv`, which backs the key-value storage operations of §4.1. A conforming host **refuses to persist** any runtime whose object names violate the rule, failing closed with prior state retained; an unvalidated name is never interpolated into SQL.

These schemas are designed by the model from the application's own requirements. A portfolio tracker receives tables for equities, trades, and dividends, because that is what a portfolio tracker is about. This is a departure from storing application data as an opaque blob: the shape is legible at rest, so both the host and the agent can reason about the application's data when modifying it later.

6.3 Materialisation and isolation

Storing every application's tables in one file raises the obvious question: what stops an application from reading tables that are not its own? The answer, shown in Figure 6, is that it never executes against that file.

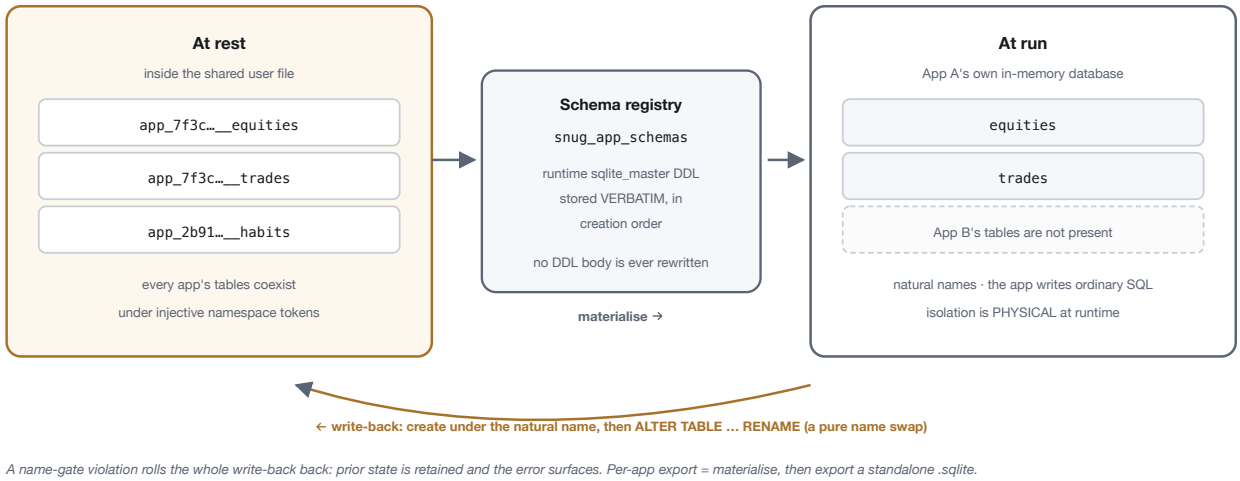

Figure 6. Materialisation. The registry stores each application's DDL verbatim; at load, those objects are replayed into a fresh in-memory database containing nothing else. The application executes ordinary SQL against natural table names, and isolation at runtime is physical rather than advisory.

At load, the host materialises the application's objects into a database belonging to that application alone, under their natural names. Application SQL executes there. Host-namespace tables and other applications' tables are not merely protected — they are absent from the database being queried. Isolation is physical at runtime, which is a stronger property than validation, because it does not depend on catching every hostile query.

Two implementation commitments make this trustworthy. **No DDL body is ever rewritten.** The registry stores the runtime schema verbatim — tables, indexes, triggers, and views, in creation order — and replay reproduces it exactly. At-rest names are produced by `ALTER TABLE … RENAME`, a pure name swap, rather than by editing statement text. String-rewriting SQL was considered and rejected: statement parsing is fragile and injection-prone, and the embedded engine exposes no authorizer with which to validate the result.

Per-application export follows directly: materialise, then export. The user receives a standalone `.sqlite` file with natural table names — their application's data, in a form with no dependency on Snug at all.

6.4 Host obligations

A conforming host must satisfy four obligations, which together are what make the file portable in practice rather than in principle.

1. **Never require its backend for execution.** The client is static files; application reads and writes hit the browser's copy of the user database.
2. **Offer export and import.** One-click download and upload of the canonical file. Default export strips secrets and vacuums; including them is an explicit opt-in. On import, the file's endpoint settings are treated as executable configuration and require user re-confirmation before any agent turn runs — an imported file is untrusted input like any other.
3. **Optionally act as a synchronisation origin**, using conditional requests for concurrency: retrieval returns bytes with a revision tag; a write carries the expected revision and is rejected on mismatch, with a missing precondition rejected outright. Cookie authentication requires a double-submit CSRF token, and

cross-origin policy is fail-closed. First login provisions the user record only — a host never creates an empty database image that could overwrite local state; the client pushes up.

4. **Support pluggable origins** through an information/pull/push contract, so that a user may keep their file in storage they control. Conflicts are resolved by compare-and-swap on the revision token; divergence is surfaced to the user, and last-writer-wins occurs only on explicit user action.

The third obligation's final clause is small and load-bearing. A host that provisions an empty file on first login will eventually overwrite somebody's real data with emptiness, because a fresh login on a new device is indistinguishable from a first-ever login. Pushing up rather than provisioning down makes the failure impossible.

6.5 Execution modes

Three modes are defined, summarised in Figure 7, and the user chooses between them in settings.

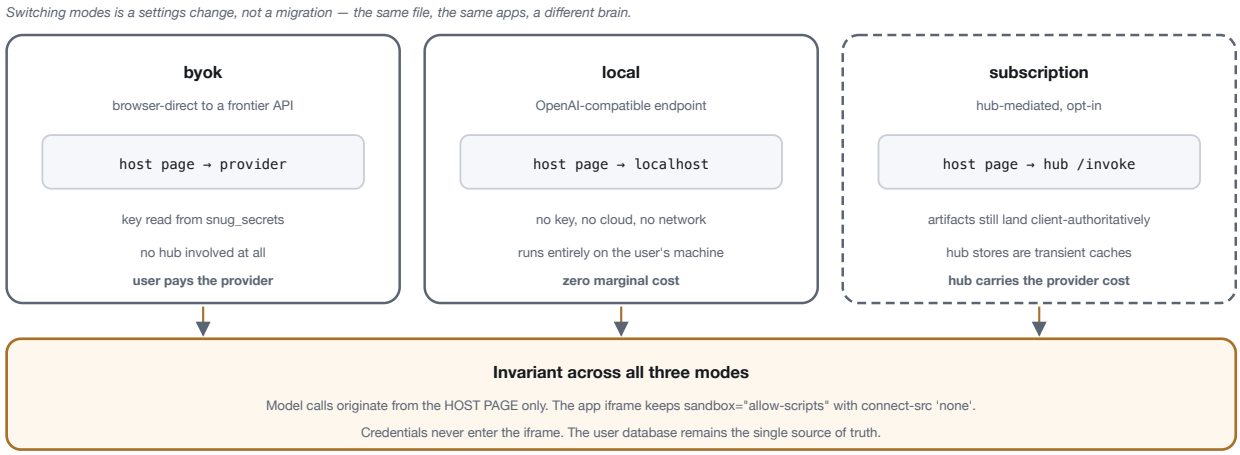

Figure 7. Execution modes. The three differ in who pays and where inference happens, and in nothing else that matters: the same file, the same applications, the same security constraints. Local mode runs with no key, no cloud, and no network dependency at all.

In every mode the user database remains authoritative. In subscription mode a host may cache artifacts and conversation history server-side, but the client fetches artifact content and writes it into the user database itself; host-side stores are transient caches. This is what keeps mode-switching a settings change rather than a migration — and what ensures that a host's disappearance costs the user a service, not their software.

7 Conformance

An implementation may claim conformance with Snug v0.1 if it satisfies the following. The list is deliberately testable; each item is asserted by the reference implementation's suite.

7.1 Hosts

- Validate every inbound frame against the published schemas, accepting unknown fields (R2) and rejecting unsupported versions with `UNSUPPORTED_VERSION` (R1).

- Mint `instanceId`, deliver it in `snug:host-ready`, and route by message source, never by `appId` (R4).
- Emit exactly one terminal `snug:app-response` per accepted `requestId` (R3).
- Enforce the size, string, budget, and backoff limits of R6.
- Strip the five credential headers from application-originated requests at the envelope boundary (C1).
- Run application frames with `sandbox="allow-scripts"` only and `connect-src` blocked (C2).
- Parse agent replies with the graduated tolerance of §4.5 and convert failure to `PARSE_FAILED` with `rawExcerpt` and `attemptsRemaining`.
- Execute application code with no configured model, for applications that request none (G1).

7.2 Applications and SDKs

- Announce on mount and wait for `snug:host-ready` before issuing requests.
- Echo `instanceId` on every request and use a unique `requestId` per instance.
- Ignore unknown frame types and unknown fields (R2).
- Treat streaming frames as provisional and the terminal frame as authoritative (R3).
- Never assume storage APIs are available; use the storage frames.

7.3 Hosts additionally implementing the v0.2 draft

- Carry the schema version in `PRAGMA user_version` and migrate forward-only.
- Compute namespace tokens with the normative function, and refuse to persist any runtime whose object names violate the naming rules — failing closed.
- Materialise per-application runtimes so that isolation is physical, and store DDL verbatim.
- Offer export and import; strip secrets and vacuum by default; require re-confirmation of imported endpoint settings.
- Retain at least five unpinned versions and never prune the pinned factory version.
- Never require the backend for application execution.

8 Related work

8.1 The Model Context Protocol

MCP standardises how a model reaches outward — to tools, resources, and external systems — and it does that job well enough that Snug does not attempt it. The relationship is complementary and directional, and it is the cleanest way to locate Snug in the current landscape:

	MODEL CONTEXT PROTOCOL	SNUG PROTOCOL
Direction	Agent → capability	Application → agent
Question	What can the model reach?	What can a user's application ask of the model?
Author	A developer building a server	A model, on the user's instruction
Unit	Tools and resources	Sandboxed applications and their data
Trust	Server is a configured, trusted integration	Application is hostile by default
Persistence	Out of scope	Central: one portable file

Table 3. MCP and Snug address inverse directions of the same boundary.

A host may well implement both, and we expect the common configuration to be exactly that: MCP for what the agent can do, Snug for what the user's applications can ask. The differing trust posture is the reason they are separate protocols rather than one. An MCP server is something a user or administrator deliberately installed; a Snug application is something a model wrote minutes ago from a prompt of uncertain provenance. Those two things cannot safely share a permission model.

8.2 Vendor application platforms

Applications embedded in assistant products — sandboxed frames, model-mediated, distributed through a vendor's directory — validate the category emphatically, and their architectures rhyme with this one. The difference is ownership rather than mechanism. Those applications are publisher-built, vendor-reviewed, and vendor-hosted, and the user's data resides in the vendor's cloud under the vendor's terms. That is a reasonable arrangement for professionally published software with commercial support behind it.

It is a poor fit for the case this protocol addresses: an application built by one person for themselves, which no directory would list and no reviewer should need to approve, and whose value to its owner is precisely that it is theirs. Snug is the open, user-owned counterpart — not a competing store, but the layer that lets such software exist without one.

8.3 Local-first software

The local-first tradition — Ink & Switch's articulation of it, and the broader body of work on user agency in software — supplies the values this protocol encodes: data on the user's device, no dependency on a vendor's continued existence, and real ownership rather than a licence. The essays in that tradition, and the related arguments for software as a home-cooked meal and for development practised by people who are not professional developers, describe a world in which people make small software for themselves.

What that literature largely predates is the arrival of a general-purpose authoring capability. It argued for personal software when producing it still required programming skill. The models removed that barrier and, in doing so, created the problem this specification addresses: with authoring solved, the binding constraints became isolation, custody, and portability. Snug's contribution is to answer those three at the protocol layer, so that the answer is not one vendor's implementation detail.

9 Limitations and future work

Single-host at a time. The file is authoritative and synchronisation is compare-and-swap on a revision token, which detects divergence but does not merge it. Two devices editing concurrently produce a conflict the user resolves. Convergent replicated data types would allow genuine multi-device concurrency and are the natural next step; they are a substantial addition to a format whose present virtue is that any SQLite tool can read it.

Browser-bound. The isolation model is the browser's: frame sandboxing and content security policy. This is a well-understood boundary with wide deployment, but it does bind conforming hosts to browser-class environments. A native host would need an equivalent mechanism, and the protocol does not currently specify what would qualify.

Authenticated connections are reserved, not specified. `CONSENT_REQUIRED` and `AUTH_REQUIRED` are reserved error codes for a future revision in which applications reach the user's own services. The custody doctrine that revision must satisfy is already fixed by §5.4 — credentials in the user's file, host custody never — but the wire surface is deliberately unpublished until it has been built and audited. We would rather reserve the space than specify it speculatively.

Size bounds are conservative. A 64 MiB file suits many personal applications and constrains others; a media-heavy application will meet the ceiling. Raising it interacts with synchronisation cost, since the present model transfers whole files.

No formal verification. The security properties are argued from structure and tested by suite, not proved. The isolation claim in particular — that materialisation makes cross-application reads impossible rather than merely difficult — would benefit from formal treatment.

The agent reply contract is prose-shaped. Tolerant parsing is a pragmatic response to how models behave today. As constrained decoding becomes universally available, the fallbacks in §4.5 should become dead code, and a later revision may make schema-constrained replies mandatory rather than optional.

10 Conclusion

The economics of software authorship have changed permanently, and the change is not primarily about productivity. It is that the minimum viable audience for a piece of software has fallen to one. A person can now have software that exists solely because they wanted it — which is a category that has never meaningfully existed before.

Whether that category becomes durable depends on questions models do not answer. Software built for one person must still be safe to run, since nobody reviewed it. It must be able to hold data without that data becoming somebody else's asset. It must survive its author's choice of vendor. These are the ordinary concerns of infrastructure, and they are settled by protocols or not at all.

Snug proposes a specific settlement. Treat the application as hostile and confine it absolutely; give it the model as a capability it calls rather than an engine it contains; and put everything the user accumulates into one file they can carry away. The wire protocol of §4 is normative today; the storage format of §6 is published as a draft for review. Both are MIT licensed, both are specified by schemas exported from a working

implementation, and both are offered in the expectation that other implementations will find them wanting in specific, correctable ways.

The measure of this work is not whether the reference implementation succeeds. It is whether a person can build something small and strange and entirely their own — and still have it, on their own terms, years later, on a device and a provider nobody has chosen yet.

Normative references

1. **Snug Protocol Specification, v0.1.** Frames, chat envelope, and rules R1–R6. snugprotocol/spec — `SPEC.md`
2. **Snug Protocol JSON Schemas.** Normative message shapes for all nine frame types and the chat envelope, published byte-identically from the reference implementation. snugprotocol/spec — `schemas/`
3. **Snug Protocol Specification, v0.2 (draft).** Portable User Database Format: host-namespace tables, native per-application tables, synchronisation origins, export and import. snugprotocol/spec — `SPEC-v0.2-draft.md`
4. **RFC 2119.** Key words for use in RFCs to indicate requirement levels. S. Bradner, IETF, 1997
5. **HTML Standard** — the `sandbox` attribute and its origin semantics. WHATWG
6. **Content Security Policy Level 3** — the `connect-src` directive. W3C
7. **SQLite File Format.** SQLite Consortium

AUTHOR

Jeetu Maker

SECURITY CONTACT

security@snugprotocol.org

STATUS

Wire protocol v0.1 is normative and published. The portable user database format is a v0.2 draft, published for review; details may change before it becomes normative.

Appendix A · Message schema summary

Required fields for every published message, from the normative schemas. All frames additionally require `v: 1` and `type`; validators MUST accept unknown fields (R2).

MESSAGE	REQUIRED BEYOND `V` AND `TYPE`
`snug:app-announce`	`appId`, `displayName`
`snug:host-ready`	`instanceId`, `protocolVersions`, `capabilities`, `theme`
`snug:app-message`	`requestId`, `instanceId`, `appId`, `action`
`snug:app-cancel`	`requestId`, `instanceId`
`snug:app-response` streaming	`requestId`, `ok: true`, `streaming: true`, `text`
`snug:app-response` final	`requestId`, `ok: true`, `streaming: false`, `data`

MESSAGE	REQUIRED BEYOND `v` AND `type`
`snug:app-response` error	`requestId`, `ok: false`, `error`
`snug:db-request`	`requestId`, `instanceId`, `op`, and the operation's own fields (e.g. `sql` for `exec`)
`snug:db-response`	`requestId`, `ok` — with `error` when `ok: false`
`snug:host-event`	`event`
`snug:app-event`	`event`
`chat envelope`	`snug: 1`, `appId`, `instanceId`, `requestId`, `action`

Table 4. Required fields by message type. Optional fields are described in §4.

The envelope is not a frame: it is tag-delimited text sent to an agent endpoint, detected by the `[SNUG_APP_REQUEST]` prefix together with the `snug: 1` marker, and therefore carries no `type` field.